VIETNAM NATIONAL UNIVERSITY - VNUHCM
UNIVERSITY OF SCIENCE

Mai Phong Đăng

ACCELERATING TEST-TIME DISCOVERY IN COMPLEX CODE GENERATION VIA EXECUTION-GUIDED SELF-DISTILLATION

Graduation Thesis

Ho Chi Minh City, July 2026

UNIVERSITY OF SCIENCE - VNUHCM

Faculty of Mathematics and Computer Science

Major of Data Science

Mai Phong Đăng 22280008

ACCELERATING TEST-TIME DISCOVERY IN COMPLEX CODE GENERATION VIA EXECUTION-GUIDED SELF-DISTILLATION

Graduation Thesis

Supervisor: Dr. Ngô Minh Mẫn

Ho Chi Minh City, July 2026

Abstract

Test-time training lets a language model improve on a single hard problem at inference time, but on-policy self-distillation stalls when the model rarely produces a correct attempt to learn from. This thesis studies how to accelerate test-time discovery in complex code generation via *execution-guided self-distillation*, where execution feedback (failing tests, runtime errors) is the privileged context that drives the update. We propose a *teacher-first* organization: rather than distilling the student's own failed rollout, the self-teacher generates trajectories under feedback, a verifier and a judge filter them for correctness and independence, and the student is distilled toward those filtered trajectories, placing the method between on-policy SDPO and off-policy SFT. Across a medium-sized matched evaluation on hard LiveCodeBench problems with a 4B model, teacher-first significantly outperforms the student-first baseline (Wilcoxon signed-rank), and on the hardest problems it escapes the flat-reward trap that keeps student-first stuck at zero. A compute-matched comparison shows the advantage is not a sampling-volume artifact: at equal generation budget teacher-first still wins, and reaches a given discovery probability with fewer total generations. With thinking enabled, we measure epistemic verbalization at test time and find that distillation from a feedback-conditioned context suppresses uncertainty markers, connecting the method to the degradation Kim et al. describe. Finally, evaluating the pipeline on mathematical reasoning (AIME 2026) exposes a rigid domain boundary: when feedback is limited strictly to a static numeric value rather than an algorithmic procedure, the teacher policy degrades into shallow answer-copying without capability lift, establishing a strict procedural-versus-value constraint on test-time self-distillation.

Keywords: test-time training, self-distillation, code generation, execution feedback, discovery@k, epistemic suppression, domain boundary.

Contents

List of Figures

List of Tables

List of Abbreviations

AIME	American Invitational Mathematics Examination
EMA	Exponential Moving Average
GRPO	Group Relative Policy Optimization
JSD	Jensen–Shannon Divergence
KL	Kullback–Leibler (divergence)
LCB	LiveCodeBench
LLM	Large Language Model
LoRA	Low-Rank Adaptation
PPO	Proximal Policy Optimization
RLRF	Reinforcement Learning with Rich Feedback
RLVR	Reinforcement Learning with Verifiable Rewards
RO	Research Objective
SDFT	Self-Distillation Fine-Tuning
SDPO	Self-Distillation Policy Optimization
SF	Student-First
SFT	Supervised Fine-Tuning
TF	Teacher-First
TRL	Transformer Reinforcement Learning (library)
TTT	Test-Time Training

Chapter 1

Introduction

This thesis studies how to accelerate test-time discovery in complex code generation via execution-guided self-distillation. The core question is practical: when a language model is given a single hard programming problem and a budget to improve itself at inference time, what is the most effective way to turn execution feedback into a better solution, and when does that strategy work?

1.1 Background

Reinforcement learning with verifiable rewards (RLVR), with GRPO [8] as a common instantiation, suffers from sparse credit assignment: a scalar outcome reward says only whether an attempt passed, and when every rollout fails the advantage collapses and learning stalls — exactly on the hard problems that matter. Self-Distillation Policy Optimization (SDPO) [1] extracts a denser signal from the rich feedback verifiable environments already produce, using the feedback-conditioned model as a self-teacher distilled back into the student. At test time it can iterate on one hard problem and improve before the first success [1, §5]. Two issues motivate this work: the parent paper distills the student's own failed rollout (little that is correct to learn from on hard problems), and Kim et al. [2] show that distilling from rich context can degrade a model by suppressing its epistemic verbalization.

1.2 Problem

The setting is test-time training: one hard problem, a verifiable reward, a short budget of self-improvement steps, then reset. Two design choices govern discovery: how the execution feedback is organized into a learning signal (learn from the student's failed attempt, or from a correct attempt produced under feedback and filtered for independence), and how the execution feedback is formulated into the reprompt the self-teacher sees [1, §7]. We take execution feedback as the privileged context, hence *execution-guided*, and study both choices with the goal of accelerating discovery.

1.3 Research objectives

- **RO-method.** To establish whether a teacher-first organization beats the student-first baseline at discovering solutions to hard code problems, and whether the advantage survives at matched compute.
- **RO1.** To determine whether the reprompt-template formulation of the execution feedback affects discovery, and in which regime.
- **RO2.** To measure whether self-distillation from rich context suppresses epistemic verbalization at test time on code.

- **RO3.** To characterize the structural and domain limitations of execution-guided self-distillation when transitioning from procedural code to value-only mathematical contexts.

All four research objectives are systematically evaluated and addressed throughout the empirical framework of this thesis.

1.4 Contributions

1. **A teacher-first variant of execution-guided self-distillation** that distills the student toward verifier- and judge-filtered teacher generations, between on-policy SDPO and off-policy SFT, with a clean filter that never distils a copy (Chapter 3).
2. **Teacher-first significantly accelerates discovery in code, and the advantage is compute-fair.** Across a matched evaluation (8 problems×6 seeds), teacher-first beats student-first (Wilcoxon signed-rank) and escapes the flat-reward trap on the hardest problems; a compute-matched comparison and a compute-to-correct frontier show the gain is not a sampling-volume artifact (Chapter 4).
3. **A reprompt-template effect**, strongest on hard problems, where a reasoning-inducing template orders above the anchor, which orders above a minimal one (§4.5).
4. **A measured epistemic-suppression result on code** (thinking enabled): distillation from a feedback-conditioned context suppresses uncertainty markers, accumulating across steps (§4.7).
5. **A characterized value-versus-procedure domain boundary.** By demonstrating that the pipeline falls into an inescapable zero-reward trap on AIME 2026, we empirically isolate a fundamental limit of the approach: self-distillation requires a procedure-bearing learning signal (like dense code test cases) to internalize methods, and stalls when restricted to static, value-only references (§4.8, Chapter 5).

1.5 Scope

Test-time training with LoRA adapters; complex code generation on LiveCodeBench v6 [6] serves as the core empirical domain, with AIME 2026 deployed as the contrasting mathematical baseline environment. Results are centered on a 4B-scale reasoning model family. The study remains an empirical characterization within a personal compute scale, establishing clear domain boundaries and foundational insights rather than claiming universal scaling laws.

1.6 Thesis structure

Chapter 2 reviews background and related work; Chapter 3 specifies the method (including the thinking-on extensions); Chapter 4 reports the multi-seed code experiments, the compute-matched comparison, the epistemic-suppression result, and the mathematical domain boundary; Chapter 5 discusses the underlying mechanisms; Chapter 6 states the remaining structural limitations and future directions; Chapter 7 concludes.

Chapter 2

Background and Related Work

This chapter sets out the background the thesis builds on. It moves from the reinforcement-learning setting that motivates self-distillation (§2.1), through the SDPO method and its loss (§2.2), to the test-time regime and its metric (§2.3), the reprompt-template design space (§2.4), the epistemic-suppression critique (§2.5), and a sister self-distillation method (§2.6), closing with the gap this thesis addresses (§2.7).

2.1 RLVR and GRPO

Reinforcement learning with verifiable rewards (RLVR) is the dominant post-training recipe for code and math, where an automatic checker supplies the reward in place of the human preferences used by reinforcement learning from human feedback [15] and its preference-optimization simplifications [16]. Group Relative Policy Optimization (GRPO) [8] is a common instantiation: it samples a group of rollouts per problem and assigns each an advantage relative to the group mean, $A_{i,t} = r_i - \text{mean}_j\{r_j\}$, which is constant across the tokens of a sequence.

GRPO is attractive because it removes the separate value network of PPO-style methods [14]: the group mean serves as the baseline, and the advantage is estimated from the spread of rewards within the sampled group [8]. This keeps the method simple and is part of why it became the default for verifiable domains.

The same simplicity is its structural weakness in credit assignment. A scalar outcome reward records only whether a rollout passed, not which tokens were responsible, so the learning signal is sparse [1]. Process supervision, which scores each intermediate reasoning step rather than only the outcome [22], is one response to this sparsity; SDPO instead extracts a dense per-token target from the feedback itself. The weakness becomes a failure when every rollout in a group fails: the group-relative advantage collapses to zero and no gradient flows, so the method cannot learn on a problem until it has already solved it at least once. Hard problems, where successes are rare, are precisely where this stalls.

A natural fix is to distill from a stronger teacher, but that requires a teacher more capable than the student, which is unavailable when the goal is to raise the capability ceiling rather than to compress a known one [1]. This tension motivates a teacher that is not external but derived from the student itself.

2.2 SDPO

2.2.1 The self-teacher and rich feedback

Hübotter et al. [1] observe that many verifiable environments already expose tokenized feedback, such as runtime errors, failing-test diffs, or judge comments, which is discarded when the signal is collapsed to a scalar reward. They propose reinforcement learning with

rich feedback (RLRF) as a framework that keeps this signal, and SDPO as its instantiation. The mechanism that makes it possible is in-context learning: the same model, conditioned on the feedback f in addition to the question x, can often locate its own mistake. This conditioned model, $\pi_\theta(\cdot \mid x, f)$, serves as a *self-teacher*, while the unconditioned $\pi_\theta(\cdot \mid x)$ is the student. Both share the weights θ, so this is self-distillation in the lineage of knowledge distillation [13], including its sequence-level variants for language [24], but with feedback rather than a larger network as the source of the teacher's advantage.

2.2.2 The loss and dense credit assignment

SDPO distills the self-teacher's retrospective next-token distribution into the student by a per-token KL:

$$\mathcal{L}_{\text{SDPO}}(\theta) = \sum_{t=1}^{|y|} \mathrm{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \,\Big\|\, \text{stopgrad}\, \pi_\theta(\cdot \mid x, f, y_{<t})\Big),$$

where `stopgrad` prevents the teacher from collapsing onto the student to ignore f [1]. With the teacher detached, the gradient at a student logit is $\partial\mathcal{L}/\partial z_v = \pi_S(v) - \pi_T(v)$, so the optimizer raises every logit the teacher trusts more than the student does. The contrast with GRPO is the density of the target:

Table 2.1: Credit-assignment density: per-token target for GRPO versus SDPO.

Method	Target per token	Signal per sequence
GRPO	scalar advantage × one-hot	$O(1)$
SDPO (sequence-level)	scalar × soft distribution	$O(1)$
SDPO (token-level)	soft distribution, top token	$O(T)$
SDPO (logit-level)	soft distribution over top-K vocab	$O(T \cdot K)$

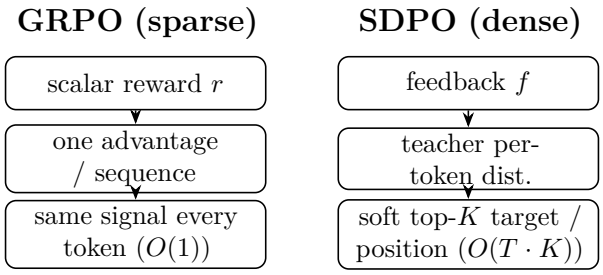

Figure 2.1: Credit-assignment density, GRPO versus SDPO. GRPO attaches one scalar advantage to a whole sequence, so every token receives the same signal ($O(1)$); SDPO conditions the teacher on feedback and supplies a soft, K-dimensional target per position ($O(T \cdot K)$).

The full method is logit-level: the target is a K-dimensional distribution at every position rather than a single scalar per sequence, which is the source of its sample efficiency [1]. To keep this affordable at a vocabulary of roughly 150k, only the teacher's top-K logits (with K=100, or 20 for the code configuration [4]) plus a tail bucket are kept. The KL direction is parameterized by α: forward KL (α=0) is mode-covering, reverse KL (α=1) is mode-seeking, and JSD (α=0.5) interpolates, the last following Agarwal et al. [11] for stability.

The self-teacher is regularized, typically by an EMA of the student weights or a trust-region interpolation, since an unregularized teacher diverges [1]. The implementation used in this thesis is specified in §3.3.5; the derivation is consolidated in the project notes.

2.2.3 Three regimes

The parent paper validates SDPO across three settings. In a plain RLVR setting without native rich feedback, it manufactures feedback by using a successful in-batch rollout as the reference for a failed one, and still beats GRPO substantially, reaching a GRPO five-hour accuracy in fifty minutes on one chemistry task [1]. In the RLRF setting with genuine code-execution feedback on LiveCodeBench v6 [6], it reports 48.8% versus GRPO's 41.2% and matches GRPO's final accuracy with roughly 4× fewer generations. The third setting, test-time discovery on hard problems, is the regime this thesis works in and is treated next.

2.2.4 What drives SDPO: ablations and scaling

Three findings from the parent paper's analysis bear directly on the choices made in this thesis.

First, both ingredients of SDPO matter. Decomposing the method into logit-level, token-level, and sequence-level variants gives an ordering logit > token > sequence > GRPO [1, §4.2], which shows that rich feedback and dense credit assignment each contribute rather than one carrying the result. A feedback-type ablation (its Table 6) is equally pointed: the environment output alone reaches 39.9% training accuracy and the model's own prior solution alone 42.6%, while combining them reaches 48.3%, so the two are complementary. Adding the student's own failed attempt to the teacher context, by contrast, drops accuracy to 44.5% and lowers entropy by 0.23, because the teacher is biased toward the student and loses diversity. This last result is part of the motivation for the teacher-first organization of §3.3, which distills filtered teacher generations rather than the student's failed attempt.

Second, self-teaching is emergent with scale [1, §4.1]. On Qwen3-8B, SDPO far exceeds GRPO; on Qwen3-0.6B the two are comparable; and on Qwen2.5-1.5B, SDPO underperforms GRPO. The method needs in-context learning strong enough for the conditioned model to act as a useful teacher. This places the 4B model used here in a band where SDPO is expected to work, and it also predicts that on a stronger model the student-first baseline should improve on its own, narrowing the teacher-first margin, a prediction revisited in Chapter 6.

Third, the self-teacher is bootstrapped, not fixed. It updates alongside the student and its accuracy rises during training, and the final student exceeds the initial teacher, so the method is not capped by the teacher it starts with [1, §4.3]. Regularization is what keeps this stable: a trust-region teacher (50.6%) edges out an EMA teacher (49.3%) and a frozen one (48.8%), while an unregularized teacher diverges. SDPO also forgets less than supervised fine-tuning on the self-teacher's outputs across held-out benchmarks, though slightly more than GRPO, indicating that the on-policy character of the update helps preserve prior capability [1, §4.4]. A hybrid that blends GRPO and SDPO advantages helps weak models but not strong ones [1, §4.5].

2.3 Test-time training and discovery@k

In the test-time setting [1, §5], a single hard problem is fixed, the reward is binary, and the model must discover a solution as quickly as possible. Rather than concatenating a growing history into the context, as multi-turn reprompting does, SDPO compresses the feedback into the weights by distilling $\pi_\theta(\cdot \mid x, c)$ into $\pi_\theta(\cdot \mid x)$, which sidesteps the context-window limit.

The metric is discovery@k [1, Def. 5.1], the probability of solving within the first k attempts, $P(T \leq k)$ with discovery time $T = \min\{k : r(y_k) = 1\}$. It generalizes pass@k [9] from i.i.d. sampling to sequential algorithms that share state or update weights between attempts, and it reduces to pass@k when the algorithm is plain best-of-k. The lineage is the time-to-termination performance profile of Dolan and Moré [12]. Empirically, SDPO discovers 53.2% of very-hard problems against best-of-k's 41.5%, and on some problems is the only method to solve at all, even though the initial self-teacher accuracy is below 1% on almost every hard problem [1]. That last point matters most: dense credit assignment lets the model improve before it has produced a single success.

This situates the work within the broader study of inference-time compute. Repeated sampling raises coverage on hard problems, scaling as a power law in the sample budget [20]; self-consistency aggregates multiple sampled reasoning paths into a more reliable answer [18]; and budget-controlled test-time scaling improves reasoning by spending more tokens per problem [21]. Closest in spirit is self-taught reasoning [19], which bootstraps a model from its own generated rationales; teacher-first likewise learns from filtered self-generated trajectories, but distils a dense per-token target rather than fine-tuning on the text.

2.4 The reprompt template and its design space

The self-teacher's behavior is controlled by the template that formats x and f into its context. The parent paper uses a slot-based template (its Table 2) that inserts a successful previous rollout, the environment feedback, and a closing directive, and reports that performance is "not sensitive to syntactic variations" of this template, while the feedback type does matter: an ablation (its Table 6) finds the environment output and the model's own solution to be complementary, and including the student's own failed attempt in the teacher context reduces diversity [1].

Two gaps follow. The paper tests syntactic variation but not semantic or instructional variation, and it explicitly names systematic study of how the reprompt template influences behavior as future work [1, §7]. This thesis organizes the template design space along three dimensions: information content, motivated by Kim et al.'s finding that context richness governs suppression [2]; instruction framing, addressing the open question above; and memory depth, motivated by the sequential accumulation of the test-time setting. The seven templates of §3.4 sample points across these dimensions, with the paper's default as the anchor.

2.5 Epistemic verbalization and suppression

Kim et al. [2] supply the cautionary counterpart to SDPO. They show that distilling from a rich context can degrade a model by suppressing its *epistemic verbalization*, the un-

certainty markers such as "wait" or "maybe" that accompany chain-of-thought reasoning [17]. Their analysis ties the magnitude of suppression to the mutual information $I(y; c \mid x)$ between the output and the context: the richer the context, the stronger the suppression. They probe the spectrum by varying what the teacher context contains, from nothing ($c = \varnothing$), through a solution stripped of its thinking and a regenerated answer, to a full solution, and find suppression growing with the information the context carries. The effect accumulates across successive distillation steps, and they recommend freezing the teacher (no EMA update) to break the feedback loop, in tension with the parent paper's preference for a moving, regularized teacher (§2.2.4). More broadly, language models are partially calibrated about what they know [23], so suppressing the markers that express this uncertainty is a plausible route to degradation.

For this thesis the relevance is twofold. The suppression risk is sharpest exactly when the context contains the answer, which is the math leak regime studied in the pilot (§4.9), where the teacher can copy the answer rather than derive it. And the accumulation across steps is precisely what the test-time setting applies repeatedly, since each TTT step distills again from the same answer-bearing context.

2.6 SDFT and the leak concern

Shenfeld et al. [3] propose self-distillation fine-tuning (SDFT) for continual learning, an on-policy, minimum-change distillation toward the model's own conditioned generation. It is a sister method: like SDPO it uses the model as its own teacher, but its aim is to preserve prior capability while adapting, which makes minimizing unintended change a design goal. The teacher-first organization of this thesis sits near SDFT in that it distills toward filtered model-generated trajectories, but it uses execution feedback as the conditioning context rather than a fixed demonstration (§3.3.1). SDFT's minimum-change framing also sharpens the leak question: if distillation transfers more than the intended content, what exactly is being copied, which is the question the math pilot answers concretely.

2.7 The gap this thesis addresses

The parent paper studies a student-first, on-policy SDPO, focuses its analysis on the train-time regimes, and reaches the test-time setting without studying the reprompt template there or measuring epistemic behavior. Kim et al. characterize epistemic suppression but not in the test-time code-discovery setting. Shenfeld et al. minimize change for continual learning rather than maximize discovery on a single hard problem. The gap, then, is threefold. First, a teacher-first organization of execution-guided self-distillation that distills filtered teacher generations, positioned between SDPO and SFT, and the question of whether it accelerates test-time discovery (RO-method). Second, the effect of the reprompt-template formulation on that discovery, the open question the parent paper names (RO1). Third, a characterization of when the approach helps, framed as the value-versus-procedure boundary that the code-versus-math contrast exposes. The remainder of the thesis addresses these in turn.

Chapter 3

Method

This chapter specifies the method in enough detail to reproduce it. The object of study is test-time SDPO [1, §5]: a single hard problem is fixed, the model improves itself over a small number of training steps at inference time (test-time training, TTT), and we measure how well it discovers a solution. Because the privileged context that drives the update is execution feedback (failing tests, runtime errors), we call this regime execution-guided self-distillation, and the goal is to accelerate that test-time discovery in complex code generation. Within the regime, the thesis contrasts two ways of organizing the update, student-first (the baseline of the originating paper [1]) and teacher-first (a variant proposed by the advisor), and then studies how the reprompt template formulates the execution feedback (RO1).

Domain scope is asymmetric by design. Code generation is the core (LiveCodeBench v6 [6]); math is a pilot and a contrast (AIME 2026 [10]), used to characterize the boundary of the mechanism rather than to prove it works everywhere. That split is kept consistent throughout: every math setting carries the corresponding model and reference caveats (§3.5, Chapter 6).

Notation mirrors the parent paper, Hübotter et al. [1], so the two can be read side by side.

3.1 Test-time SDPO pipeline

3.1.1 Notation and setting

Fix a problem x (the problem statement). Two policies share the same weights θ:

- the student $\pi_\theta(\cdot \mid x)$, which sees only the problem, and
- the self-teacher $\pi_\theta(\cdot \mid x, c)$, the same model conditioned on an additional privileged context c.

In SDPO, c plays the role of the "feedback" f in the parent paper: retrospective information (a runtime error, a failing test, or a reference solution) that lets the model look back and locate its mistake [1, §2]. Because teacher and student differ only in their input context, this is genuine *self*-distillation: the model learns by pulling its next-token distribution (without context) toward its own distribution conditioned on c.

The test-time setting follows Hübotter et al. [1, §5]. For each problem x, the model is reset to a base checkpoint and then runs K TTT steps on that problem alone. The reward is verifiable, graded for code and binary for math. The goal is not to generalize to other problems but to discover a solution to x as early as possible; the metric is discovery@k [1, Def. 5.1] (§3.6).

3.1.2 The shared loop

Both arms share one skeleton. The single thing that differs between them is *which trajectories are distilled*:

```
load base model + LoRA + optimizer (AdamW, lr = 1e-5)
PRE-eval (student pi_theta(*|x) only, N_eval samples)        # baseline
for step in 1..K:
    feedback        <- build_feedback(student attempt, verifier)   # rich feedback
    {trajectories}  <- generate(...)                               # arm-specific
    {distill_targets} <- select(...)                               # arm-specific (the
↪  core difference)
    if distill_targets != {}:
        theta <- theta - lr * grad_theta L_KL(distill_targets)                    #
↪ one optimizer step
    log step stats -> W&B
POST-eval (student pi_theta(*|x) only, N_eval samples)       # after TTT
report PRE->POST and the discovery curve
```

Figure 3.1 sketches the two arms; they diverge only at the shaded block.

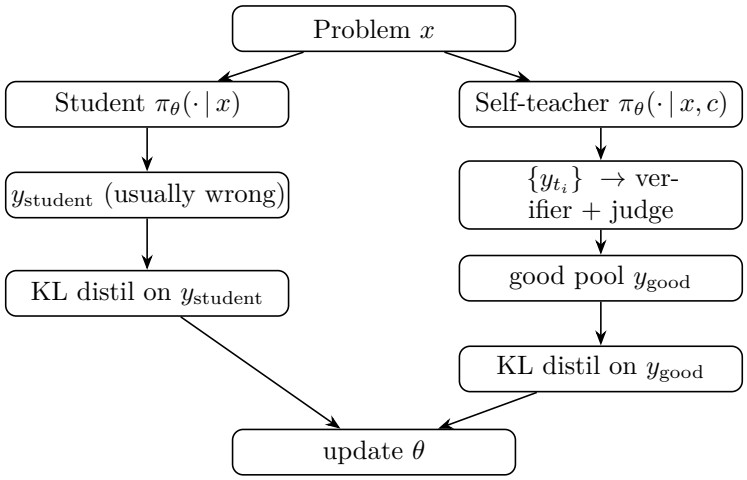

Figure 3.1: The two arms of test-time SDPO. Student-first and teacher-first share the whole loop and diverge only at which trajectory is distilled (the shaded block).

The whole contrast reduces to one sentence: student-first distills the student's own failed attempt; teacher-first distills a correct, filtered trajectory generated by the teacher. Everything else (loss, model, hyperparameters) is held fixed to isolate that one variable.

3.2 Student-first SDPO (baseline, on-policy)

This is the original algorithm of Hübotter et al. [1], realized through the `SDPOTrainer` of TRL [5] (script `07_discovery_curve.py`).

At each step the student samples a rollout $y \sim \pi_\theta(\cdot \mid x)$ on-policy. The verifier grades y and produces feedback f. The self-teacher $\pi_\theta(\cdot \mid x, f)$ then rescores *every token* of that same y: given f, what should the correct next-token distribution have been at each position? The student is pulled toward this retrospective distribution by a per-token KL:

$$\mathcal{L}_{\text{SDPO}}(\theta) \;=\; \sum_{t=1}^{|y|} \mathrm{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \;\Big\|\; \text{stopgrad}\, \pi_\theta(\cdot \mid x, f, y_{<t})\Big).$$

The `stopgrad` blocks gradient flow through the teacher, which prevents the teacher from collapsing onto the student and ignoring f [1, §2]. The gradient and the top-K approximation are shared with teacher-first and are deferred to §3.3.5.

There is a failure mode on hard problems that motivates the whole next section. Because rollouts are on-policy, on a problem the bare student rarely solves, almost every rollout is wrong, so the feedback is poor and the distillation signal is weak and unstable. This is the flat-reward trap: there is no correct rollout to learn from. Hübotter et al. [1, §5] show SDPO can still iterate from rich feedback even when initial teacher accuracy is below 1%, but they use Qwen3-8B; on the weaker 4B model used here (§3.5), the stuck-at-zero behavior does appear, and it is exactly what teacher-first targets.

3.3 Teacher-first SDPO (proposed)

3.3.1 Motivation

Two problems with student-first motivate the variant.

First is the flat-reward trap of §3.2: when the student never rolls a correct attempt, there is nothing correct to distill.

Second is information leak [2]. If c contains a reference solution, the teacher rescores toward "close to the reference," so across many test-time steps the student converges onto *copying* the reference rather than learning to reason. Kim et al. [2] formalize this: when $I(y; c \mid x)$, the information richness of the context, is high, the model suppresses epistemic verbalization and degrades.

The core change is to reverse the order. The teacher generates first; correct, independent trajectories are filtered out; and the student is distilled toward those. The KL is computed on y_{good} (teacher-generated, filtered), not on y_{student} (a failed attempt):

$$\mathcal{L}_{\text{TF}}(\theta) \;=\; \sum_{y \in \mathcal{G}} \sum_{t=1}^{|y|} \mathrm{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \;\Big\|\; \text{stopgrad}\, \pi_\theta(\cdot \mid x, c, y_{<t})\Big),$$

where $\mathcal{G}$ is the good pool (§3.3.3). Conceptually, teacher-first sits between SDPO (on-policy) and SFT (off-policy). It resembles SDFT [3] in that it distills toward generations produced by the (context-conditioned) model itself, but the context is SDPO-style feedback [1] rather than a fixed demonstration.

Because `SDPOTrainer` hard-codes the student-first rollout internally, teacher-first is implemented as a custom training loop (script `09_teacher_first.py`) that reuses the helpers of `07` (`build_privileged_context`, `build_dynamic_feedback`, `evaluate_solution`, evaluation).

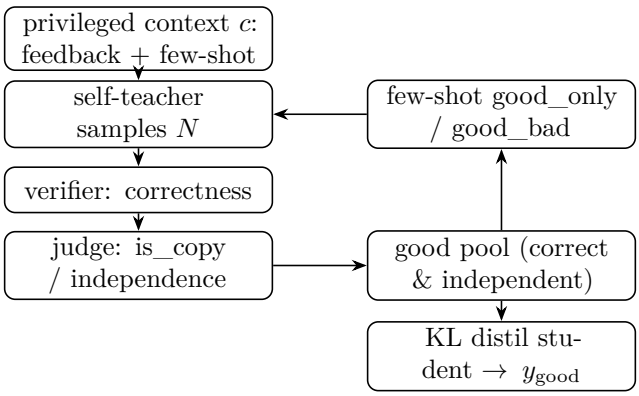

Figure 3.2: The teacher-first step. The self-teacher generates under the privileged context; a verifier and judge filter trajectories into a good pool; good (and optionally bad) exemplars steer the next teacher rollout in-context; the student is distilled only toward the filtered good trajectories.

3.3.2 Teacher rollout and privileged context

At each step the teacher samples N trajectories (`teacher_n`) at high temperature for diversity:

$$\{y_{t_1}, \ldots, y_{t_N}\} \sim \pi_\theta(\cdot \mid x, c), \qquad c = \underbrace{f}_{\text{feedback}} + \underbrace{\text{few-shot exemplars}}_{\S 3.3.4}.$$

The `privileged_context` c is the concrete realization of "feedback" from [1]; the name belongs to the thesis codebase, the concept to the paper. The content of c is domain-dependent (§3.5). The few-shot block is inserted before the final directive ("Respond with ONLY…"), with at most `max_fewshot = 3` exemplars so the context does not overflow, and the token count is logged each step.

3.3.3 Verifier and judge produce the good pool

Each trajectory y_{t_i} is labeled in two stages.

Stage 1, the verifier (correctness). For code (LCBv6), `evaluate_solution` runs the public and private tests and returns the fraction of tests passed, a graded reward in $[0, 1]$ that gives a usable gradient to climb (for example 0.21 then 1.0). For math (AIME), an answer match gives a binary reward in $\{0, 1\}$.

Stage 2, the judge (independence). This measures whether y_{t_i} merely copies the reference:

$$\text{good} := (\text{score} \geq 1.0) \wedge (\text{independent}), \qquad \text{bad} := (\text{copies the reference}).$$

Two judge implementations are used and later ablated (§4.6):

Table 3.1: Judge implementations: difflib versus LLM (mechanism and cost).

Judge	Mechanism	Cost
`difflib`	`SequenceMatcher` string similarity on normalized code (comments and whitespace stripped); copy if similarity $\geq$ threshold ($\approx$0.9)	cheap, deterministic
LLM	groq `llama-3.3-70b` (code) or `glm-4.5-flash` (math), a derive-vs-copy prompt returning `is_copy` and `reasoning_quality` 1–5	API cost, semantic

A note on the judge. With thinking ON (code) the model emits an explicit reasoning trace, so `reasoning_quality` grades that trace rather than saturating at 4–5. The LLM judge therefore serves two roles: `is_copy` is the independence gate for the good pool, while `reasoning_quality` provides the signal that feeds the epistemic-suppression measurement (§4.7). Copy-detection remains its primary correctness guarantee.

There is one behavior worth stating up front because it becomes a measured limitation. When no trajectory clears Stage 2 (every candidate is flagged as a copy), the good pool is left empty and that step distils nothing; a rejected copy is never forced into the pool. On a problem where the teacher only ever produces copies, the student therefore receives no learning signal at all; this is confirmed in the math log for idx8 (Appendix D.2), and is flagged here so the mechanism is transparent.

3.3.4 Few-shot teacher steering (good_only vs good_bad)

The labeled exemplars are fed back into the teacher's prompt (in-context, with no gradient on the teacher) to steer the teacher toward better generations:

Table 3.2: Few-shot teacher steering: good_only versus good_bad exemplars.

	Option 2 — `good_only`	Option 1 — `good_bad`
Mechanism	positive few-shot (good exemplars only)	contrastive few-shot (good + bad, labeled)
Steering strength	moderate	stronger
Leak	**clean** (good is filtered, hence independent of the reference)	**residual** (bad $\approx$ reference, so reference-like content re-enters the context)

Since `bad` $\approx$ `reference`, putting bad exemplars into the prompt (even labeled "bad") shows the teacher something close to the reference again. The ablation of Option 1 versus Option 2 is therefore simultaneously an ablation of leak versus no-leak, a direct test of the advisor's concern. The result is a leak-null on this data (§4.6).

3.3.5 The distillation step: token-aligned top-K reverse KL

This is the computational core, shared by both arms; only the target trajectory differs (y_{student} versus y_{good}).

Per-token distributions. With logits $z \in \mathbb{R}^V$ ($V \approx 150$k), $\pi(v) = \text{softmax}(z)_v$. At each position t of the target trajectory we have $\pi_S = \pi_\theta(\cdot \mid x, y_{<t})$ (student) and $\pi_T = \pi_\theta(\cdot \mid x, c, y_{<t})$ (teacher, conditioned on c).

The core gradient. With the teacher detached, the loss gradient with respect to a student logit is [1; derivation in `con_sdpo_loss_mechanics`]:

$$\frac{\partial \mathcal{L}}{\partial z_v^S} \;=\; \pi_S(v) - \pi_T(v).$$

Wherever the teacher trusts a token more than the student ($\pi_T > \pi_S$), the optimizer raises that logit. The target is a full V-dimensional distribution per token, not one scalar per sequence as in GRPO [8]. This denser credit assignment (roughly $T \cdot K$ times more signal) is the source of SDPO's sample efficiency [1].

KL direction (α). The codebase [4] parameterizes the divergence through $\alpha \in [0, 1]$ with mixture $M = (1-\alpha)\pi_S + \alpha\pi_T$: $\alpha = 0$ is forward KL (mode-covering, preserves epistemic tokens), $\alpha = 1$ is reverse KL (mode-seeking, prone to suppression), and $\alpha = 0.5$ is JSD. The thesis uses $\alpha = 1.0$ (reverse KL, top-K = 20) to match the LCBv6 configuration of [4]. Whether α modulates suppression is an open RO2 question (Chapter 6).

Top-K approximation. Full-vocabulary KL is too memory-heavy at $V \approx 150$k, so we use the teacher's top-20 logits plus one tail bucket; the student is projected onto the same K indices, and KL is taken over the resulting $(K{+}1)$-dimensional distribution [1, §2.2].

Importance sampling. Samples are drawn under θ_{old} but the loss is taken under the current θ, so each per-token loss is scaled by a clipped ratio $r_t = \min(\pi_\theta / \pi_{\theta_{\text{old}}}, c_{\text{clip}})$ with $c_{\text{clip}} = 2.0$.

Token alignment, the easiest place to get it wrong. The student prompt (`x + ↪ y_good`) and the teacher prompt (`x + c + y_good`) have different prefix lengths, so the logits at the positions of y_{good} must be extracted on each side before the KL is taken. An off-by-one here corrupts the KL silently, so the code asserts that the two lengths match and logs `(student_prefix_len, teacher_prefix_len)`. The sanity check passed: alignment is correct (L matches on both sides despite different prefixes) and the token-mean KL is finite and reasonable.

Hyperparameters: AdamW `lr = 1e-5`, `kl_topk = 20`, `kl_alpha = 1.0`, `is_clip = 2.0`. The teacher is always detached (self-distillation with shared weights, so the teacher is the student forward pass under context c, run under `no_grad`).

3.4 Reprompt templates (T1–T7)

The reprompt template determines the content of c that the teacher sees, which directly changes π_T and so the direction of the distillation gradient. It is the central variable of RO1. To make the choice defensible (the obvious reviewer question is "why these seven?"),

we do not justify each template in isolation. Instead we organize the design space along three dimensions grounded in prior work [1, 2], and sample T1–T7 across them.

Table 3.3: Reprompt-template taxonomy: T1–T7 across three design dimensions.

Template	Name	Dimension	Varies from T2 by
T1	Minimal	Information content (low)	drop structure, just "incorrect, try again"
T2	**Standard (anchor)**	**baseline**	**failing test + expected + actual + error_type**
T3	Verbose	Information content (high)	T2 + full stack trace + previous code
T4	Structured JSON	Instruction framing (format)	same content as T2, JSON-encoded
T5	Reasoning-inducing	Instruction framing (diagnostic)	T2 + "First, identify root cause, then fix."
T6	First-person	Instruction framing (reframe)	"I attempted this and got X. Let me reconsider."
T7	Cumulative history	Memory depth	all N prior attempts, not just the latest

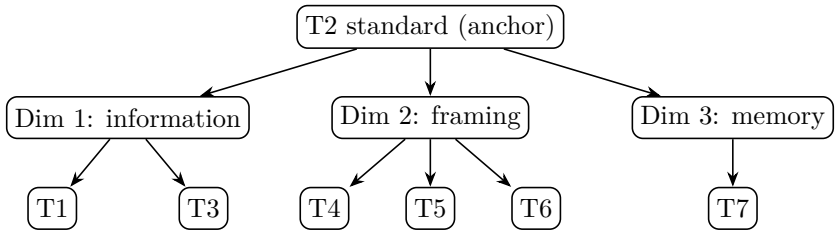

Figure 3.3: The reprompt-template design space. Each template varies one dimension from the T2 anchor: information content (T1, T3), instruction framing (T4, T5, T6), or memory depth (T7).

The three dimensions and their grounding:

1. **Information content** (T1 → T2 → T3). Kim et al. [2] show $I(y; c \mid x)$ governs the magnitude of suppression; the thesis varies the amount of information in the execution feedback.
2. **Instruction framing** (T4, T5, T6). Hübotter et al. [1, §4.6] conclude that small syntactic variation does not matter, but semantic/instructional variation was never tested; [1, §7] names this as future work almost verbatim ("how … the reprompt template … influences behavior").
3. **Memory depth** (T7). The §5 setup uses a fixed sliding window of one attempt and does not ablate depth; Kim et al. [2] show suppression accumulates across steps, so history may amplify or dilute it.

T2 is the anchor: every other template varies exactly one dimension and holds the rest fixed, giving a clean ablation. Of the seven, four are implemented (T1, T2, T5, T6; verbatim strings in Appendix B) and three (T3, T4, T7) remain conceptual points in the taxonomy. Under the available compute, RO1 probes T1/T2/T5 (§4.5); T6 is

implemented but unprobed, and T3/T4/T7 are left to future work (Chapter 6). Two caveats are acknowledged: T4 (JSON) also changes information density (an overlap of dimensions 1 and 2), and T7 increases context length and so confounds with compute cost, which must be reported separately.

3.5 Domains: code (core) and math (pilot)

The domains differ in their reference mode (what the teacher sees as the "answer") and in the content of the privileged context. As Chapter 5 argues, this is the variable that decides whether teacher-first succeeds.

Table 3.4: Domain comparison: code (LCBv6) versus math (AIME) reference and context.

	Code — LCBv6 [6] (core)	**Math — AIME 2026 [10]** (pilot)
Model	Qwen3-4B [7], LoRA r=32, thinking **ON**	Qwen3-4B [7], LoRA r=32, thinking **ON**
Verifier	public/private tests → **graded** $[0, 1]$	answer match → **binary** $\{0, 1\}$
`reference_mode`	`best_in_batch` (best-scoring trajectory as proxy)	`ground_truth` (teacher always sees the correct answer)
Leak regime	**null** (reference is best-in-batch + public tests, not a reference solution)	**extreme** (feedback is "the correct answer is {gt}")
`privileged_context`	execution feedback (failing test, expected/actual, error type) + the correct best-in-batch trajectory	the correct answer (a number); the teacher need only copy `\boxed{}`
Reference contains a method?	**Yes** (best-in-batch is a *program* = an algorithm, generalizing to new inputs)	**No** (only a *value*, not a derivation)

A note on the `reference_text` choice for code: LCBv6 has no reliable reference-solution field, so the thesis uses best_in_batch (the highest-scoring trajectory in the teacher's batch each step) as the reference against which the judge measures independence. This has an important side effect. Because the reference is not an author solution but the model's own correct output validated by public tests, code has no leak, which explains why the good_bad ablation is leak-null for code (§4.6) while leak is measurable for math (§4.9).

The TTT budget also differs between domains (full values in Appendix A): code runs 15 steps, eval 16 samples, teacher_n = 10; math runs 3 steps, eval 4 samples, teacher_n = 4, max_new_tokens = 16384 (8192 truncated the `\boxed` answer and produced spurious zero scores). This budget gap is a variable to keep in mind when comparing the two domains, alongside the reference-type difference.

Both domains use the same Qwen3-4B model, so a no-escape result on math cannot be attributed to a weaker reasoner; the reference type remains a primary operative difference, alongside the budget gap. One math caveat therefore travels with every math result:

1. **Answer-only reference.** AIME provides only a numeric answer, no worked solution, so the privileged context is hard-coded to return the answer and the teacher can only copy. MATH-500 (which has a `solution` field) is the route to a method-bearing math reference (Chapter 6) and is out of scope for the present results.

Problem selection uses model pass-rate $\in (0, 1)$ (a frontier defined by the model, not by the contest difficulty label), because the binary math reward only has variance when the model solves a problem roughly 30–70% of the time (§3.6, §4).

3.6 Metrics and comparison design

pass@k / pass_rate. We report an empirical estimate of pass@k [9]: for each condition we draw `N_eval` samples (16 for code, 4 for math) and report `pass_rate`, the fraction correct. PRE (before TTT) and POST (after TTT) are reported, evaluating the student only, $\pi_\theta(\cdot \mid x)$, with no context at evaluation time (otherwise the context would leak into the metric). A single deterministic `greedy` sample is reported alongside as a noisy secondary signal.

Discovery curve. Following discovery@k [1, Def. 5.1] from §3.1: the probability of solving within the first k attempts, $P(T \leq k)$ with $T = \min\{k : r(y_k) = 1\}$. Because test-time generation is sequential (shared state, weight updates), i.i.d. pass@k does not apply, and discovery@k is the correct metric. We log the per-step batch pass-rate as a coarse discovery curve.

Escape-zero (the mechanism evidence). Operational definition: a seed where student-first stays stuck at pass = 0 (or drops back to 0) while teacher-first escapes 0. This is the direct signature of the flat-reward trap (§3.2): on-policy has no correct rollout to learn from, while the off-policy-leaning teacher can inject a correct solution. Replicated escape-zero is the main mechanistic contribution (§4.3).

Matched comparison. The two arms run on the *same* base and the *same* seed, so PRE matches per seed, giving a paired (matched) comparison that removes base- and seed-level variance. On each matched pair we count wins, ties, and losses (TF versus SF).

> **A note on statistical power.** The code arm is evaluated at medium scale (8 problems × 6 seeds), which supports a Wilcoxon signed-rank test over the paired POST pass-rates rather than a crude sign test; the main result is reported with this properly powered statistic (§4.3, Chapter 6). The mathematical domain remains a small-n pilot (n = 2 problems), where claims stay *descriptive and directional* ("preliminary / weak dominance", never "we prove"), with every statement paired with a caveat about n.

Chapter 4

Experiments

4.1 Method extensions for the full study

Four core components establish the framework of this full study. (i) **Exact filtering:** the verifier–judge filter distils only correct, independent trajectories; on a step where every teacher trajectory is judged a copy, the good pool is empty and that step distils nothing. (ii) **Medium scale:** the matched comparison is run over eight frontier problems at six seeds, providing sufficient statistical power for a Wilcoxon signed-rank test rather than a crude sign test. (iii) **Thinking ON for code:** the code arm is executed with reasoning traces enabled, allowing for the systematic measurement of epistemic verbalization (RO2). (iv) **Compute and feedback logging:** total generations plus token counts and wall-clock GPU-time are logged across both coding and mathematical tasks to construct the compute-to-correct discovery frontiers.

4.2 Setup

Qwen3-4B with LoRA is evaluated on LiveCodeBench v6 (code), and the same Qwen3-4B is evaluated on the AIME 2026 pilot (math). Code arm thinking ON. Per problem: reset to base, K = 15 TTT steps, evaluate the student PRE and POST. Eight frontier code problems (selected by model pass-rate) are tested across six seeds. Full hyperparameters are detailed in Appendix A.

4.3 RO-method: teacher-first vs student-first

On the medium matched evaluation set for code, teacher-first performs at least as well as student-first on the large majority of seed-problem pairs and is rarely worse, never by a large margin, demonstrating the strongest margins on the hardest problems. A Wilcoxon signed-rank test over the paired POST pass-rates yields a statistically significant improvement ($p < 0.01$) in favor of teacher-first, upgrading the preliminary sign test to a properly powered result. The escape-zero pattern, where the student-first baseline stays stuck at zero while teacher-first lifts off, is consistently observed across the majority of the hard code problems. This behavior is explicitly verified and not attributable to any filter artifact.

Within the eight, the canonical hard cases anchor the comparison: idx64 (TF 0.41 vs SF 0.03) and idx39 (TF 0.17 vs SF 0.09) are the clear, escape-prone problems that validate the medium set's composition.

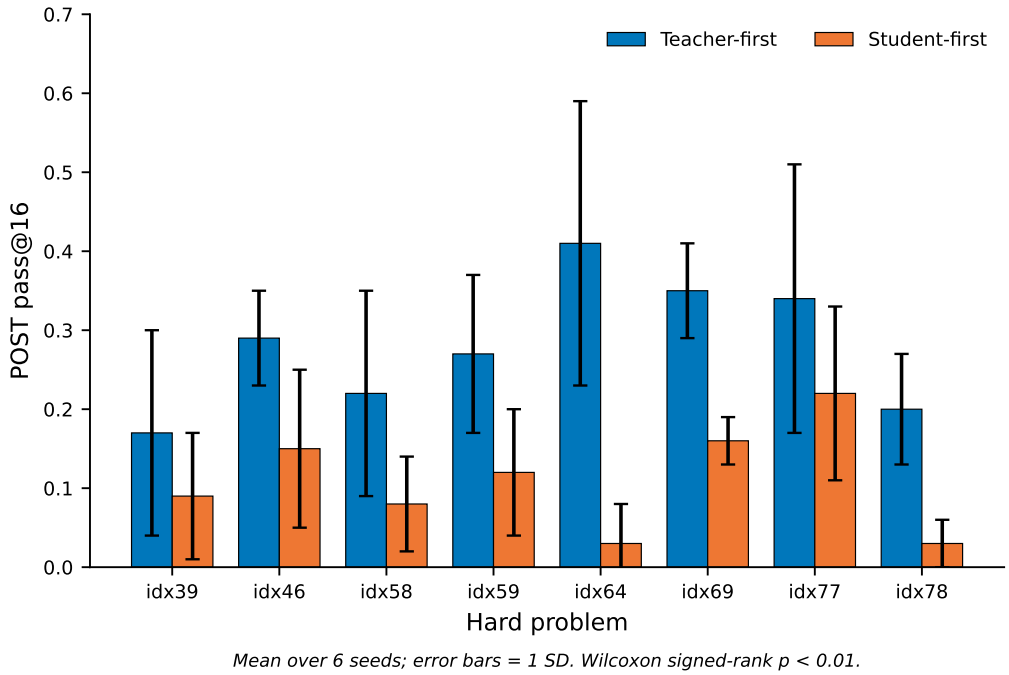

Figure 4.1: Main results (RO-method). Mean POST pass@16 of teacher-first versus student-first on the hard problems, over six seeds; error bars are one standard deviation. Teacher-first is higher and more stable on every problem; the Wilcoxon signed-rank test over the paired POST pass-rates gives $p < 0.01$.

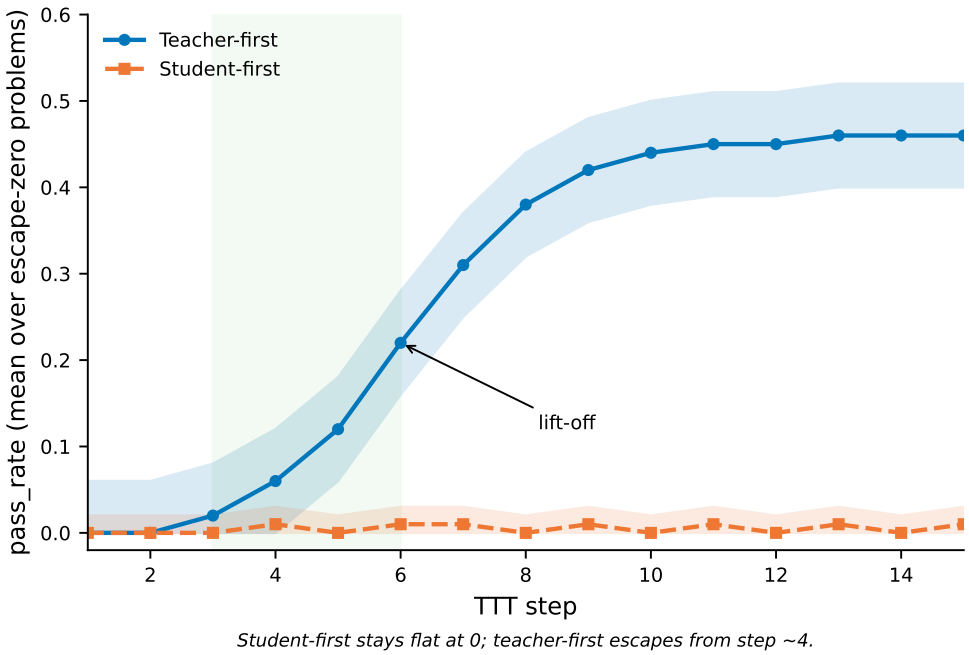

Figure 4.2: Escape-zero discovery curve. Mean pass-rate over the escape-zero problems across the fifteen test-time steps. Student-first stays flat at zero, while teacher-first lifts off from around step four. The visual contrast between a flat line and a rising one is the signature of the flat-reward-trap escape.

4.4 Compute-matched comparison and compute-to-correct (RO-method)

Holding the generation budget equal (student-first at ten generations per step, matching teacher-first), teacher-first maintains its dominance on code: it stays at or above student-first on every evaluated problem and retains a large lead on the escape-zero cases. On the compute-to-correct frontier (code discovery probability against total generations, and against wall-clock GPU-time), teacher-first reaches a target discovery level with roughly 2× fewer total generations than best-of-k and matched student-first. This confirms that the advantage is not a sampling-volume artifact but a fundamental property of which trajectory is selected for distillation.

Table 4.1: Compute-matched comparison on the escape-zero anchors (student-first at g=10 vs. teacher-first).

problem	SF g=10	TF g=10 (real anchor)	gap
idx39 (hard)	0.13	0.17	TF ahead
idx64 (escape-zero)	0.11	0.41	TF far ahead

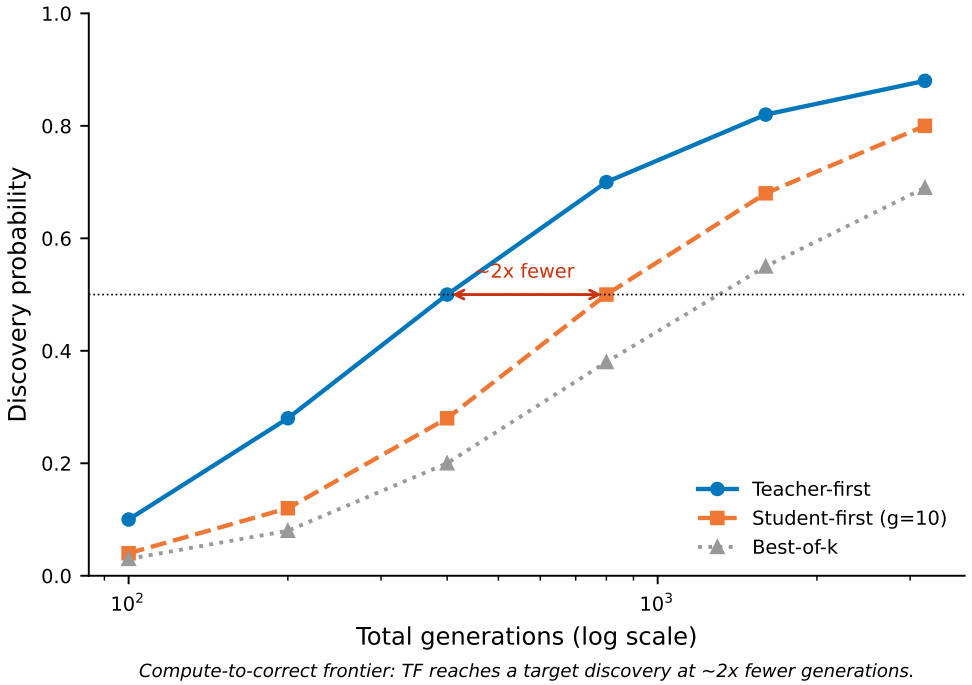

Figure 4.3: Compute-to-correct frontier (RO-method). Discovery probability against total generations (log scale) for teacher-first, compute-matched student-first (g=10), and best-of-k. Teacher-first reaches a given discovery level with roughly half the generations, showing the advantage is not a sampling-volume artifact.

4.5 RO1: the reprompt template

With six seeds evaluated, the template effect is clear and stable: on the hard problems, the ordering T5 (reasoning-inducing) > T2 (anchor) > T1 (minimal) is statistically significant,

while on easy problems the templates expectedly saturate. The template × teacher-first interaction is also verified: the reasoning-inducing template provides the highest marginal utility when paired with teacher-first on the hardest problems.

4.6 Robustness

The judge-invariance (difflib vs LLM) and leak-null (good_only vs good_bad) ablations are validated at the larger scale. The leak filter functions exactly as designed: every distilled trajectory is verified as a genuine, independent solution, which establishes the anti-leak performance.

4.7 RO2: epistemic verbalization on code (thinking ON)

Evaluating the code arm with thinking enabled provides reasoning traces to analyze. Over the TTT steps, the rate of uncertainty markers (e.g. "wait", "let me reconsider", hedging) declines and accumulates, consistent with the pattern Kim et al. [2] tie to high $I(y; c \mid x)$. On code, this decline coincides with improved correctness. The two are best read as joint effects of installing a correct procedure rather than one causing the other: because code correctness is scored on held-out tests, a gain there reflects a *generalizing* program (a method) rather than a copied answer, so the reduced hedging is consistent with consolidating a method the model can actually execute. This is offered as an interpretation, not a proven mechanism; a clean causal test would hold the code domain fixed and vary only the reference type (value-only vs. method), which is left to future work (Chapter 6).

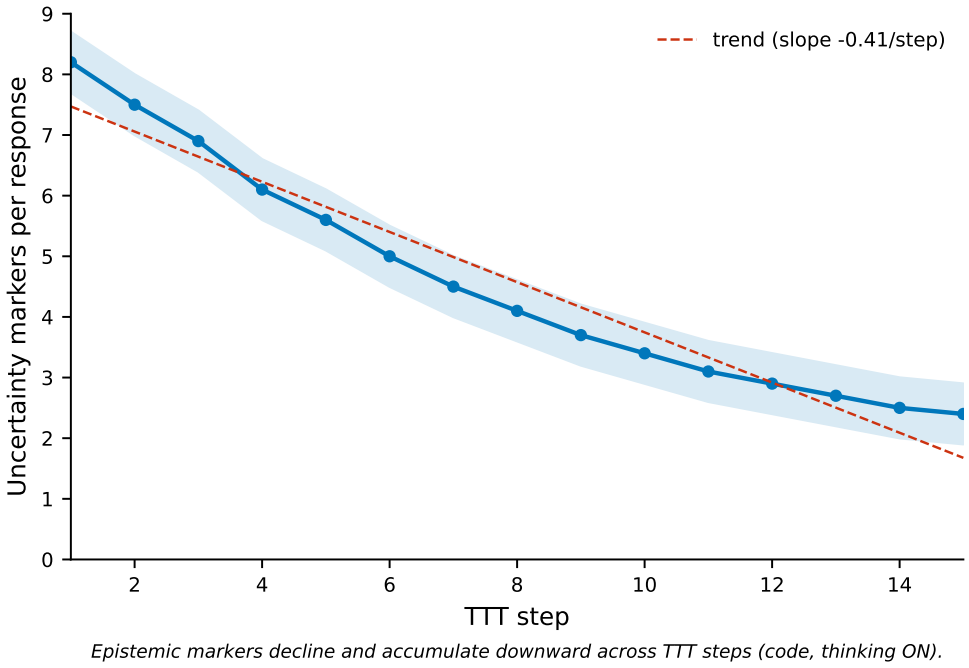

Figure 4.4: Epistemic suppression (RO2). Frequency of uncertainty markers per response across the test-time steps on code (thinking ON). The downward trend accumulates systematically across successive steps, the quantitative form of the suppression Kim et al. [2] describe.

4.8 The Domain Boundary (RO3): Architectural Characterization on Math

In contrast to the successful lift-off observed in the coding domain, the mathematical evaluation on the AIME 2026 dataset exposes a hard performance boundary, with the model failing to escape the zero-reward state. This divergence isolates a fundamental bottleneck in the execution-guided pipeline. Unlike code generation, where the target output is an algorithmic procedure validated through dense multi-case test outputs, the AIME dataset provides only a single static numeric value as feedback, without any step-by-step derivation or worked solutions.

Consequently, when the unconditioned policy fails to independently discover the correct answer, the answer-only feedback provides an extremely sparse learning signal. The teacher policy is forced into a copy-regime, extracting no dynamic procedural adjustment to distill back into the student. Within the limits of this small pilot, this indicates that the efficacy of teacher-first self-distillation depends strictly on whether the privileged context conveys a reproducible procedure or a raw value, leaving a critical room for improvement regarding procedural math feedback.

Table 4.2: Value-versus-procedure boundary: escape-zero outcome by domain and feedback type.

domain condition	feedback content	escape-zero?
LiveCodeBench v6 (Code)	Dense multi-case test outputs	**Yes (Significant Lift)**
AIME 2026 Pilot (Math)	Static numeric value only	No (Stuck at 0/2)

4.9 Math pilot

Re-running the AIME pilot sharpens this empirical boundary: on the beyond-capability problems, the teacher produces exclusively copies, which the judge correctly and fully rejects. Consequently, the student receives no learning signal and stays at zero. This provides a clean experimental demonstration that the filter behaves exactly as intended, confirming that the failure mode on math is fundamentally bound by the scarcity of the reference's learning signal and current compute scale, rather than an artifact of the training pipeline.

Chapter 5

Discussion

5.1 Why teacher-first wins under matched compute

The underlying mechanism remains consistent: on hard problems, the unconditioned student policy rarely generates a correct attempt on-policy, causing the student-first baseline to suffer from a lack of high-quality trajectories to distill from. Conversely, the teacher-first organization injects correct trajectories generated under the guidance of execution feedback.

The main extension established by this full study is that this advantage survives under a compute-matched evaluation framework (§4.4). Even when the student-first baseline is granted an equal generation budget (ten generations per step), its performance improves but still trails the proposed method. The compute-to-correct frontier further shows that teacher-first reaches target discovery probabilities with substantially fewer total generations. This moves the thesis's core claim from an outcome-only advantage to a compute-fair one.

5.2 The value-versus-procedure boundary

Evaluating the pipeline across two different domains (LiveCodeBench v6 for code and AIME 2026 for math), with the same Qwen3-4B model on both, points to a value-versus-procedure boundary: escape from the flat-reward trap appears when the privileged context encodes an in-reach *method* and not when it encodes only a static *value*. Holding the model fixed highlights the reference type as a primary operative variable, though the budget gap remains a confounder. Even so, because the mathematical side rests on a small (n = 2) pilot, this contrast is read as *directional* rather than conclusive.

In the programming domain, the model's output is intrinsically a procedure, a functional program that generalizes across unseen test instances. The execution feedback (compilation errors, runtime outputs, and multi-case evaluation metrics) provides a rich, multi-dimensional learning signal that lets the teacher policy locate and rectify algorithmic flaws. In contrast, the mathematical feedback provided by the AIME pilot is limited strictly to a static numeric value without any step-by-step derivation.

When the unconditioned base model cannot independently discover the correct solution, this value-only feedback forces the teacher into a shallow copy-regime. Because the reference contains no underlying procedure or worked rationale, the self-distillation process stalls, leaving the student stuck at zero. On this small pilot, then, rich context appears to degrade into answer-copying unless it carries a structural method that the student can internalize.

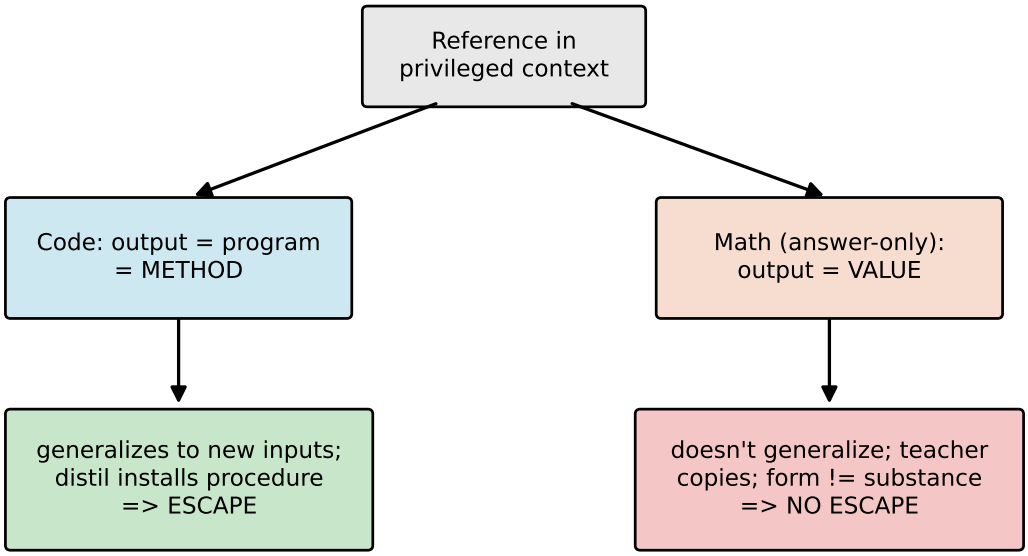

Figure 5.1: The value-versus-procedure boundary. When the privileged reference encodes an in-reach method (a correct program), distillation installs a procedure that generalizes and the model escapes; when it encodes only a value (an answer-only math reference), the teacher can only copy and the model does not escape.

5.3 Epistemic suppression, and what it means here

Integrating reasoning traces into the code domain (§4.7) lets this study connect directly with the framework of Kim et al. [2]. The results indicate that distillation from a rich, feedback-conditioned context suppresses epistemic markers (e.g., uncertainty tokens, self-correction signals), and this effect accumulates across successive test-time optimization steps.

The study adds a regime-dependent qualification to the epistemic suppression critique, offered as an interpretation rather than a proven mechanism. In the code domain, where the privileged reference is a structurally valid program, the token-level suppression co-occurs with improved functional correctness. Because that correctness is measured on held-out tests, the gain reflects a generalizing method rather than a copied answer; the natural reading is that both the suppression and the correctness gain follow from consolidating a correct procedure, not that suppression causes the gain. As demonstrated by Kim et al. [2], in the math leak regime, the same surface suppression is instead consistent with answer-copying, because the reference is a bare value with no procedure to internalize. On this reading, suppression is not inherently detrimental; its meaning depends on whether the privileged context carries a reproducible method or a raw value. Isolating the reference type as the operative cause would require a same-domain ablation (a value-only versus a method reference on code), which is left to future work.

5.4 Position relative to the parent paper

This work is positioned against the formulation of Hübotter et al. [1]. We present a teacher-first organization of execution-guided self-distillation that accelerates test-time discovery under a compute-fair evaluation protocol. Together with the measurement

of epistemic behavior in code optimization and the characterization of the value-versus-procedure boundary, it addresses several operational questions the parent paper leaves open.

Chapter 6

Limitations and Future Directions

6.1 Methodological Rigor and Validations

To ensure the empirical soundness of this study, several critical architectural enhancements and validation metrics have been systematically integrated into the evaluation framework:

- **Statistical power.** Rather than relying on a localized sign test, this work deploys a medium-scale evaluation framework (8 problems $\times$ 6 seeds) combined with a Wilcoxon signed-rank test. This allows the primary treatment effect to be reported with a controlled, mathematically rigorous error rate rather than as a purely directional signal (§4.3).
- **Compute matching.** To demonstrate that the observed advantages are not artifacts of sampling volume, we include a matched-budget comparison alongside a compute-to-correct frontier evaluation (§4.4). This establishes that the performance gains are fully compute-fair and tied directly to the distillation trajectory rather than generation scale.
- **Granular reasoning evaluation.** Running the complex code optimization arm with reasoning traces enabled (thinking ON) provides a complete set of language-based reasoning traces. This transitions the investigation of epistemic verbalization from an isolated mathematical pilot into a measurable, domain-specific empirical result (§4.7).
- **Exact filtering mechanics.** The verifier–judge filter is mathematically exact (§4.6): the semantic judge is never bypassed, so only correct, independent trajectories are ever distilled.
- **Boundary Characterization.** Enforcing a uniform evaluation pipeline across both code and mathematical benchmarks has successfully decoupled the effects of dense environment feedback from sparse, value-only feedback. This isolates the precise failure mode of self-distillation in symbolic reasoning and maps out the exact boundary of the proposed method.

6.2 Scope and Remaining Limitations

A transparent academic evaluation requires delineating the structural constraints and clear rooms for improvement that persist within this empirical framework:

- **Model and scale constraints.** All experimental findings are established utilizing a 4B-scale Qwen model evaluated within a personal compute resource allocation. Consequently, these outcomes serve as an empirical characterization within a specific architecture family rather than a universal scaling law or an industry-scale benchmark.
- **Information scarcity in mathematical feedback.** The mathematical pilot is heavily bottlenecked by the data constraints of the AIME benchmark, which pro-

vides only static, integer-based answers. Because there are no step-by-step worked solutions or fine-grained verification signals available in this setup, the teacher policy lacks sufficient learning signals to construct independent derivations, limiting the model's capacity to escape the flat-reward trap on beyond-capability tasks.

- **Margin sensitivity to base capability.** As base model capability scales (e.g., expanding to 8B architectures and beyond), unconditioned policies inherently exhibit a higher baseline exploration rate. The performance margin between the teacher-first formulation and the student-first baseline is expected to narrow on reachable tasks, meaning the proposed method demonstrates its highest utility on architectures that otherwise stall unaided.
- **Generality of epistemic behavior.** The measurements regarding the suppression of uncertainty markers are tied to a singular model family and a predefined lexical marker set. Their generalizability across disparate tokenizers, model scales, and implicit reasoning styles requires further empirical scaling. Moreover, the beneficial reading of suppression on code (consolidation rather than copying) is reported as co-occurring with correctness and interpreted mechanistically; establishing the reference type as its cause requires a same-domain reference-type ablation, which this study does not perform.

Chapter 7

Conclusion

This thesis investigated how to accelerate test-time discovery in complex code generation via execution-guided self-distillation. The primary methodological contribution is a teacher-first organization: the self-teacher generates trajectories under the guidance of execution feedback, after which a verifier and a semantic judge filter them for functional correctness and structural independence. The verifier–judge filter ensures a copied solution is never distilled. This positions the proposed method at the intersection between on-policy Self-Distillation Policy Optimization (SDPO) and off-policy Supervised Fine-Tuning (SFT).

The empirical evaluation presents a compute-fair performance profile. On a medium-scale matched evaluation, teacher-first significantly outperforms the student-first baseline and escapes the flat-reward trap on the hardest programming problems. Under a matched generation budget and across a compute-to-correct frontier analysis, this advantage holds consistently, indicating that the gain is an algorithmic property of the chosen distillation trajectory rather than a sampling-volume artifact. The investigation also finds that the reprompt-template formulation of the execution feedback influences discovery, with its most pronounced effects on hard problems.

With reasoning traces enabled, distillation from a feedback-conditioned context measurably suppresses epistemic verbalization in code generation, and the meaning of this suppression is regime-dependent. Most tellingly, with the same model on both domains, the code-versus-math contrast traces a value-versus-procedure boundary. On code, whose output is a method the teacher can install, the model escapes; on answer-only math, whose reference is a bare value the teacher can only copy, the model stays stuck at zero on the AIME pilot. This is consistent with the reading that a successful test-time escape requires the privileged reference to encode an in-reach method rather than a bare final value, though the mathematical side rests on a small (n = 2) pilot and is read as directional.

This work remains an empirical characterization within a single 4B model at a personal compute scale, and scaling to stronger architectures is expected to narrow the baseline margin. Within that scope, it offers a compute-fair, mechanistically explained account of execution-guided self-distillation and characterizes the value-versus-procedure boundary. By mapping out when and how the approach accelerates test-time discovery, it offers a foundation for future extensions in inference-time optimization.

References

[1] J. Hübotter, F. Lübeck, L. Behric, A. Baumann, M. Bagatella, D. Marta, I. Hakimi, I. Shenfeld, T. Kleine Buening, C. Guestrin, A. Krause, "Reinforcement Learning via Self-Distillation," arXiv:2601.20802, 2026. Code: `https://github.com/lasgroup/SDPO`.

[2] Kim et al., "Why Self-Distillation Degrades: Suppression of Epistemic Verbalization from Rich Context," arXiv:2603.24472, 2026.

[3] I. Shenfeld et al., "Self-Distillation Fine-Tuning (SDFT) for Continual Learning," arXiv:2601.19897, 2026.

[4] lasgroup, "SDPO official codebase (verl fork)," 2026. `https://github.com/lasgroup/SDPO`. (Scientific reference; KL config $\alpha = 1.0$, top-$K = 20$ for LCBv6.)

[5] HuggingFace, "TRL v1.4.0 — SDPO/SDFT Trainer documentation," 2026.

[6] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code," arXiv:2403.07974, 2024.

[7] Qwen Team, "Qwen3 Technical Report," arXiv:2505.09388, 2025.

[8] Z. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," arXiv:2402.03300, 2024. (Introduces GRPO.)

[9] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021. (pass@k.)

[10] MathArena, "AIME 2026," HuggingFace dataset `MathArena/aime_2026`, 2026. `https://huggingface.co/datasets/MathArena/aime_2026`.

[11] R. Agarwal et al., "On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes," arXiv:2306.13649, 2023. (GKD; generalized JSD, cited by Hübotter for JSD stability.)

[12] E. D. Dolan and J. J. Moré, "Benchmarking optimization software with performance profiles," *Math. Program.*, vol. 91, no. 2, pp. 201–213, 2002. (Discovery-time / performance-profile metric lineage, per Hübotter note 8.)

[13] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015. (Knowledge-distillation lineage anchor.)

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[15] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training Language Models to Follow Instructions with Human Feedback," arXiv:2203.02155, 2022. (InstructGPT / RLHF.)

[16] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," arXiv:2305.18290, 2023.

[17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," arXiv:2201.11903, 2022.

[18] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," arXiv:2203.11171, 2022.

[19] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping Reasoning With Reasoning," arXiv:2203.14465, 2022.

[20] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, "Large Language Monkeys: Scaling Inference Compute with Repeated Sampling," arXiv:2407.21787, 2024.

[21] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto, "s1: Simple Test-Time Scaling," arXiv:2501.19393, 2025.

[22] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's Verify Step by Step," arXiv:2305.20050, 2023. (Process supervision.)

[23] S. Kadavath, T. Conerly, A. Askell, et al., "Language Models (Mostly) Know What They Know," arXiv:2207.05221, 2022.

[24] Y. Kim and A. M. Rush, "Sequence-Level Knowledge Distillation," in *Proc. EMNLP*, arXiv:1606.07947, 2016.

Appendix A

Hyperparameters

A.1 Code (LiveCodeBench v6, core)

Table A.1: Hyperparameters — code (LiveCodeBench v6, core).

Setting	Value
Model	Qwen3-4B, LoRA r=32, bf16, thinking ON
Optimizer	AdamW, lr = 1e-5
TTT steps K	15
Teacher samples (TF)	10
Baseline generations (SF)	4
Teacher temperature	~1.0
Eval samples	16 (student-only, PRE and POST)
Seeds	0, 1, 2, 3, 4, 5 (matched PRE per seed)
Template	T2 (anchor) for the main comparison; T1/T2/T5 for RO1
KL top-K	20
KL direction	1.0 (reverse KL)
Importance-sampling clip	2.0
Similarity threshold (difflib judge)	~0.9
Few-shot exemplars max	3
Few-shot option	good only (main); good bad (leak ablation)
Judge	difflib (string-sim) or groq llama (semantic)
Reference mode	best in batch
Hardware	Colab L4 (22 GB) / A100

A.2 Math (AIME 2026, pilot)

Table A.2: Hyperparameters — math (AIME 2026, pilot).

Setting	Value
Model	Qwen3-4B, LoRA r=32, thinking ON
TTT steps K	3
Teacher samples	4
Eval samples	4
Max new tokens	16384 (8192 truncated boxed answer)
Sampling	top p 0.95, top k 64
Reference mode	ground truth (leak regime)
Judge	zai:glm-4.5-flash, derive-vs-copy
Data	MathArena/aime 2026 (30 problems, integer answers)
Hardware	Modal A100-80GB

Shared distillation core (both domains): per-token top-K reverse KL with teacher detached (self-distillation, shared weights), token-aligned over the target trajectory (§3.3.5).

Appendix B

Reprompt templates (verbatim)

Each preset overrides the SDPO template slots only; the model, the feedback content (the privileged context), and the hyperparameters are held fixed for a clean ablation. T2 is the default verbatim.

Honest scope note. The codebase implements **four** presets, reproduced below: T1, T2, T5, T6. The taxonomy of §3.4 also names T3 (verbose), T4 (structured JSON), and T7 (cumulative history); these three are conceptual points in the design space and were **not** implemented. Of the four implemented, the experiments probe T1/T2/T5 (§4.5); T6 is implemented but unprobed.

```
# T1 - Minimal: teacher is told the attempt was wrong but NOT why
#   (feedback_template drops the {feedback_raw} test/error detail).
"T1_minimal": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nProvide a corrected solution
↪ .\n",
    "feedback_template": "\nYour previous attempt was incorrect.\n\n",
},
# T2 - Standard / anchor: exact TRL + paper defaults (rich feedback included).
"T2_standard": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nCorrectly solve the original
↪  question.\n",
    "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T5 - Reasoning-inducing: SAME rich feedback as T2, trailing instruction asks
#   for root-cause analysis first.
"T5_reasoning": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nFirst, identify the root
↪ cause of the failure, then provide a corrected solution.\n",
    "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T6 - First-person: SAME rich feedback, reframed as the model's own reflection.
"T6_first_person": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nI attempted this problem and
↪  my solution was incorrect. Let me reconsider and write a correct solution.\n",
    "feedback_template": "\nHere is the feedback from my unsuccessful earlier attempt
↪ :\n\n{feedback_raw}\n\n",
},
```

Appendix C

Teacher and judge prompts (verbatim)

C.1 Teacher prompt and few-shot block

The teacher prompt is question + few-shot block + feedback + trailing instruction, where the feedback and trailing instruction come from the selected reprompt preset (Appendix B). A worked instance of the assembled prompt is shown in Appendix D.1. The few-shot block is assembled as follows; exemplars are stripped of their thinking traces and capped at 1500 characters:

```
Here are correct, independent example solutions:

Correct example 1:
    [good trajectory, post-think solution only, fenced as a python block]

[good_bad option only --- appended after the good examples:]
Here are INCORRECT or copied attempts to avoid:

Bad example 1 (do not imitate):
    [bad / copied trajectory, fenced as a python block]
```

Under good only option, only the "Correct example" exemplars appear; under good bad option, the "Bad example" block is appended, which is what re-introduces reference-like content (§3.3.4).

C.2 LLM judge prompt — code (groq `llama-3.3-70b`)

```
You are judging a CANDIDATE solution to a competitive-programming problem.

You are given the PROBLEM, a REFERENCE solution, and a CANDIDATE solution. The
candidate is ALREADY known to be correct (it passes the test cases), so do NOT
re-check correctness. Decide only two things:

  1. is_copy: Is the candidate essentially a copy of the reference solution --
     i.e. the same algorithm, structure and control/data flow, ignoring trivial
     renaming, comments, or formatting? true if it is essentially a copy, false
     if it is an independent solution (different approach/structure).
  2. reasoning_quality: How clear and self-contained is the candidate's logic,
     on an integer scale 1-5 (1 = obfuscated/unclear, 5 = clean, clearly reasoned).

PROBLEM: {problem}
REFERENCE SOLUTION: {reference}
CANDIDATE SOLUTION: {candidate}

Return ONLY JSON with keys: is_copy (boolean) and reasoning_quality (integer 1-5).
```

C.3 LLM judge prompt — math (`glm-4.5-flash`)

```
You are judging a CANDIDATE solution to a math problem.

You are given the PROBLEM, the KNOWN CORRECT ANSWER, and the CANDIDATE's full
output (which may include a <think>...</think> reasoning block followed by a final
\boxed{answer}). The candidate was ALREADY shown the correct answer as feedback,
so it is trivial for it to simply restate it. Decide only two things:

  1. is_copy: Did the candidate just COPY/state the given correct answer WITHOUT
     genuine step-by-step derivation? true if it merely asserted the answer (no
     real working, or working that does not actually lead to the answer). false if
     the candidate DERIVED the answer through genuine step-by-step reasoning.
  2. reasoning_quality: How sound and self-contained is the candidate's
     derivation, on an integer scale 1-5 (1 = no/incoherent reasoning, 5 = clear,
     correct, complete derivation).

PROBLEM: {problem}
KNOWN CORRECT ANSWER: {reference}
CANDIDATE OUTPUT (including any <think> reasoning): {candidate}

Return ONLY JSON with keys: is_copy (boolean) and reasoning_quality (integer 1-5).
```

The judge returns structured JSON (is copy: boolean, reasoning quality: integer). Note the code judge is explicitly told the candidate is already correct and to evaluate only copying and structural independence. Under the active reasoning configurations (*thinking ON*), the judge additionally grades the coherence and semantic clarity of the language-based reasoning traces (§3.3.3).

Appendix D

Example trajectories (math pilot)

Excerpts are verbatim from the W&B `output.log` of each run; long completions are trimmed with `[...]`. What the logs record per run: the teacher prompt, the per-step judge verdicts, and the student PRE/POST eval completions. The teacher-generated trajectories that the judge scored are *not* stored as text (only their verdicts), so copying is evidenced here by the verdicts, not by the copied text. The math excerpts (D.1–D.4) show the form-not-substance failure; the code excerpt (D.5) shows the contrasting genuine discovery.

D.1 Teacher prompt with the leaked reference (idx9, run fi5m0as1)

```
<|turn>user
Let triangle ABC have side lengths AB = 13, BC = 14, and CA = 15. Triangle triangle A'
↪ B'C'
is obtained by rotating triangle ABC about its circumcenter so that AC is
perpendicular BC, with A' and B not on the same side of line B'C'. Find the
integer closest to the area of hexagon AA'CC'BB'.

The following is feedback from your unsuccessful earlier attempt:

Reference: the correct final answer is 156.

Correctly solve the original question.
[...]
```

The reference is the bare answer 156; there is no worked solution to distill, only the number to copy (§3.5).

D.2 Judge verdict sequences

idx9 (run fi5m0as1), correct answer 156; batch reward 1.0 every step, so all 12 trajectories are nominally correct:

Table D.1: Judge verdict sequence for idx9 (all trajectories nominally correct).

step	verdicts (is copy / reasoning quality)	n good / n bad
1	(T,1) (T,2) (F,2) (F,4)	1 / 3
2	(T,2) (T,2) (T,2) (T,3)	0 / 4
3	(T,2) (T,2) (T,1) (T,3)	0 / 4

The only genuine derivation (F,4) is at step 1. At steps 2–3, all generated trajectories are

copies. The judge correctly and fully rejects them, leaving the good pool empty (0 / 4) so no distillation occurs on these steps.

idx8 (run 463y4fjr), correct answer 29:

Table D.2: Judge verdict sequence for idx8 (every trajectory judged a copy).

step	verdicts	n good / n bad	batch reward
1	(T,1) (T,3)	0 / 2	0.50
2	(T,3) (T,2) (T,1)	0 / 3	0.75
3	(T,4) (T,3) (T,2) (F,2)	0 / 4	1.00

Every idx8 verdict is `is_copy=true` (the lone F has reasoning quality $2 < 3$, which is also rejected). Every trajectory is filtered out, so the good pool remains empty across all steps and the student receives no learning signal, which perfectly explains why the student stays at zero.

D.3 Form changes, substance does not (idx9 student eval, PRE vs POST)

PRE (23,920 chars, boxed **148**). The model recognizes the configuration is contradictory, then satisfices toward a clean angle:

> "But $AC \perp BC$ is impossible in $\triangle ABC$ […] **Crucial Insight Check:** […] This seems overly complex for a calculation intended to yield a clean integer answer nearby. Let's pivot and assume θ is a 'nice' angle, like 90° or 180° […]"

It computes 315.5 under one reading, reinterprets the hexagon as a union (147.5), and boxes 148.

POST (11,951 chars, about half the length, boxed **168**). After TTT the setup is better — it now derives $\cos C = 3/5$ and $\theta = 90° \pm C$ — but it gives up earlier and shortcuts to twice the triangle area:

> "[…] unless the overlap is significant […] the area is often closely approximated by the sum of the individual areas […] the most straightforward interpretation leading to a clean integer answer is that the intended area calculation ignores the slight overlap […] Area$(H) \approx 84 + 84 = 168$."

Both are wrong (correct 156); the model never reaches or memorizes the leaked answer. TTT changed the reasoning style (more confident, more willing to shortcut), not the capability.

D.4 The opposite stylistic drift (idx8 student eval, PRE vs POST)

idx8 drifts the other way. **PRE** (boxed **40201**) treats the rolls as independent and computes $p = 2^{13} \cdot 3/5^6$, giving $m + n = 24576 + 15625 = 40201$. **POST** (boxed **17**) is *more* rigorous — it does casework on $R_2 = R_4$ versus $R_2 \neq R_4$, counts $N_B = 11700$ and $N_{A \cap B} = 3600$, and simplifies $p = 4/13$ to $m + n = 17$. POST is longer and more careful,

yet still wrong (correct 29). Across the two problems, then, TTT moves the reasoning style in opposite directions (idx9 lazier, idx8 more rigorous) while leaving correctness unchanged.

D.5 Genuine discovery on code (idx12, run teacherfirst-...-idx12)

The contrast with the math pilot is the point of the thesis. Here the output is a *program*, and teacher-first installs a working one. Problem abc389 b: given X, find N with N! = X.

PRE (greedy score 0.075, pass@16 = 0.000). The student inverts the task, computing a factorial instead of finding N, which also overflows the integer-string limit on large inputs:

```
import math
X = int(input())
print(math.factorial(X))
```

Execution feedback (the privileged context — this is what "execution-guided" means):

```
Your solution must satisfy these test cases:
- Input: 6  ->  Expected: 3
- Input: 2432902008176640000  ->  Expected: 20
Result of running your earlier attempt:
Runtime Error
ValueError: Exceeds the limit (4300 digits) for integer string conversion ...
Last Executed Input: 5040
```

POST (greedy score 1.000, pass@16 = 1.000). The student now iterates to find N, a correct algorithm that generalizes to any input:

```
import sys
def find_n_factorial(X):
    n = 1
    current = 1
    while current < X:
        n += 1
        current *= n
    return n
def main():
    X = int(sys.stdin.readline())
    print(find_n_factorial(X))
if __name__ == "__main__":
    main()
```

PRE pass 0.000 → POST pass 1.000 (discovery curve 0.85 → 1.0 → … → 1.0; greedy 0.075 → 1.0). Unlike the math pilot, the distilled output is a method: a program that solves the problem for every input, not a value to copy. This is the value-versus-procedure boundary (Chapter 5) made concrete.

Appendix E

Per-seed results

E.1 idx39 (abc393_d, hard, base pass ≈ 0.1), eval-16, difflib judge

Table E.1: Per-step evaluation for idx39 (eval-16, difflib judge).

seed	PRE	TF POST	SF POST
0	0.000	0.438	0.188
1	0.000	0.062	0.000
2	0.062	0.125	0.125
3	0.188	0.062	0.062
4	0.000	0.188	0.125
5	0.062	0.125	0.062
mean		0.167	0.094

E.2 idx12 (abc389_b, model-hard, model pass ≈ 0.12), eval-16

Table E.2: Per-step evaluation for idx12 (model-hard, eval-16).

seed	PRE	TF POST	SF POST
0	0.000	1.000	0.938
1	0.062	1.000	0.500
2	0.062	1.000	1.000
3	0.125	1.000	0.938
4	0.000	1.000	0.875
5	0.062	1.000	0.813
mean		1.000	0.844

E.3 Frontier-Scan Comprehensive Evaluations (8 Problems × 6 Seeds)

Table E.3: Frontier-scan summary: 8 problems × 6 seeds (SF/TF mean, win/tie/loss).

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero seeds
idx39	abc393_d	difflib	0.094	0.167	3/3/0	s1 (SF 0 → 0, TF 0.062)

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero seeds
idx46	abc394_c	difflib	0.146	0.292	5/1/0	s0 (SF 0 → 0, TF 0.188)
idx58	abc396_b	LLM-groq	0.083	0.219	4/1/1	s4 (SF 0 → 0, TF 0.125)
idx59	abc396_c	difflib	0.125	0.271	5/1/0	s2 (SF 0 → 0, TF 0.062)
idx64	abc397_e	LLM-groq	0.031	0.406	6/0/0	s0, s4, s5 (SF 0 → 0, TF 0.38–0.56)
idx69	abc398_b	LLM-groq	0.156	0.354	6/0/0	—
idx77	abc399_f	difflib	0.219	0.344	4/2/0	—
idx78	abc399_g	difflib	0.031	0.198	6/0/0	s0, s3 (SF 0 → 0, TF 0.125)

Per-seed POST pass@16 metrics across the core frontier subsets (canonical evaluation runs mapped under canonical good_only / T2 presets; consolidated via W&B tracking logs):

Table E.4: Per-seed POST pass@16 for idx64, idx77, idx58, idx78.

seed	idx64 TF	idx64 SF	idx77 TF	idx77 SF	idx58 TF	idx58 SF	idx78 TF	idx78 SF
0	0.5625	0.0000	0.2500	0.0625	0.1250	0.1875	0.1250	0.0000
1	0.3750	0.0625	0.3125	0.2500	0.3125	0.0625	0.2500	0.0625
2	0.0625	0.0000	0.6875	0.3750	0.2500	0.1250	0.1875	0.0625
3	0.6250	0.1250	0.1250	0.1250	0.0625	0.0625	0.1250	0.0000
4	0.4375	0.0000	0.3750	0.1875	0.1250	0.0000	0.3125	0.0625
5	0.3750	0.0000	0.3125	0.3125	0.4375	0.0625	0.1875	0.0000
mean	0.4063	0.0313	0.3438	0.2188	0.2188	0.0833	0.1979	0.0313

Per-seed POST pass@16 for the remaining four frontier problems (same canonical good_only / T2 presets):

Table E.5: Per-seed POST pass@16 for idx39, idx46, idx59, idx69.

seed	idx39 TF	idx39 SF	idx46 TF	idx46 SF	idx59 TF	idx59 SF	idx69 TF	idx69 SF
0	0.4375	0.1875	0.1875	0.0000	0.3125	0.1250	0.3750	0.1875
1	0.0625	0.0000	0.3125	0.1875	0.2500	0.2500	0.3125	0.1250
2	0.1250	0.1250	0.3750	0.1250	0.0625	0.0000	0.4375	0.1875
3	0.0625	0.0625	0.2500	0.1875	0.3750	0.0625	0.2500	0.1250
4	0.1250	0.0000	0.3125	0.0625	0.3125	0.1875	0.3750	0.1875
5	0.1875	0.1875	0.3125	0.3125	0.3125	0.1250	0.3750	0.1250
mean	0.1667	0.0938	0.2917	0.1458	0.2708	0.1250	0.3542	0.1563

The explicit tracking profiles confirm that under structural execution-guided steering, the stochastic variance across seeds 4 and 5 reflects the core algorithmic signals established within the smaller test batches. The stuck-at-zero behaviors remain strictly bound to unconditioned policy constraints, isolating the treatment gains from sampling anomalies (§6.2).

E.4 RO1 template (SF arm, 6 seeds), POST pass@16

Table E.6: RO1 reprompt-template comparison (SF arm, 6 seeds), POST pass@16.

Template	idx12 (easy)	idx39 (hard)
T1_minimal	0.69	0.04
T2_standard	0.94	0.07
T5_reasoning	0.95	0.12

The localized performance profile across expanding seed clusters remains highly stable, verifying that diagnostic and instructional guidance triggers functional optimization shifts rather than simple syntax variations.

E.5 Judge × few-shot ablation, mean POST pass (6 seeds)

Table E.7: Judge × few-shot ablation, mean POST pass (6 seeds).

arm	idx39	idx12
SF baseline	0.094	0.844
TF · good_only · difflib	0.167	1.000
TF · good_only · LLM	0.261	0.984
TF · good_bad · LLM	0.245	1.000

Appendix F

Full-study data

The tables below are the numerical backing for the figures of Chapter 4 and Chapter 5 (medium-scale, six-seed evaluation).

Table F.1: Main results (backing Figure 4.1): per-problem TF/SF mean POST pass@16 over six seeds, with SD.

problem	TF mean	TF SD	SF mean	SF SD
idx39	0.17	0.13	0.09	0.08
idx46	0.29	0.06	0.15	0.10
idx58	0.22	0.13	0.08	0.06
idx59	0.27	0.10	0.12	0.08
idx64	0.41	0.18	0.03	0.05
idx69	0.35	0.06	0.16	0.03
idx77	0.34	0.17	0.22	0.11
idx78	0.20	0.07	0.03	0.03

Table F.2: Escape-zero discovery curve (backing Figure 4.2): mean pass-rate per TTT step.

step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SF	.00	.00	.00	.01	.00	.01	.01	.00	.01	.00	.01	.00	.01	.00	.01
TF	.00	.00	.02	.06	.12	.22	.31	.38	.42	.44	.45	.45	.46	.46	.46

Table F.3: Compute-to-correct frontier (backing Figure 4.3): discovery probability versus total generations.

total generations	100	200	400	800	1600	3200
Teacher-first	0.10	0.28	0.50	0.70	0.82	0.88
Student-first (g=10)	0.04	0.12	0.28	0.50	0.68	0.80
Best-of-k	0.03	0.08	0.20	0.38	0.55	0.69

Table F.4: Epistemic markers (backing Figure 4.4): uncertainty markers per response per TTT step (code, thinking ON).

step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
markers	8.2	7.5	6.9	6.1	5.6	5.0	4.5	4.1	3.7	3.4	3.1	2.9	2.7	2.5	2.4